

Protokoll - Media Rating Platform (MRP)

Student: Ábris Mező

Datum: 22.01.2025

[Link zur Git Repository](#)

1. Architektur

1.1 Grundlegende Struktur

Hauptkomponenten:

- **Server:** Verarbeitet HTTP-Anfragen
- **Handler:** Verarbeiten die Requests für verschiedene Bereiche (User, Media, Rating)
- **Service-Schicht:** Enthält die Business-Logik
- **DAO (Data Access Objects):** Kommunizieren mit der Datenbank
- **Datenbank:** PostgreSQL in Docker

1.2 Wichtige Design-Entscheidungen

Warum drei Schichten (Handler, Service, DAO)?

- Handler kümmern sich nur um HTTP (Request parsen, Response senden)
- Service hat die Logik (z.B. "darf der User das löschen?")
- DAO macht nur Datenbank-Operationen
- So ist alles getrennt und einfacher zu testen

1.3 Datenbankstruktur

Tabellen:

- `users` : Benutzer-Daten (Username, Passwort-Hash, etc.)
- `media` : Filme, Serien, Spiele
- `ratings` : Bewertungen (1-5 Sterne)
- `rating_comments` : Kommentare zu Bewertungen
- `rating_likes` : Likes für Kommentare
- `favorites` : Favoriten-Liste der User

Beziehungen:

- Ein User kann viele Media erstellen
- Ein User kann jedes Media einmal bewerten
- Ratings können mehrere Likes haben

2. Implementierte Features

2.1 User-Verwaltung

- Registrierung mit Username und Passwort
- Login gibt Token, UserId und Username zurück
- Token-Validierung bei geschützten Endpoints

2.2 Media-Verwaltung

- Media erstellen (Titel, Typ, Genre, Altersfreigabe)
- Nur der Creator kann sein Media bearbeiten/löschen
- Media suchen und filtern

2.3 Rating-System

- 1-5 Sterne Bewertung
- Ein User = eine Bewertung pro Media (kann editiert werden)
- Kommentare zu Ratings
- Likes für Kommentare
- Durchschnittsbewertung wird berechnet

2.4 Favoriten

- Media als Favorit markieren
- Favoriten-Liste anzeigen

2.5 Statistiken

- User-Profil mit Favorites
- Rating-History
- Leaderboard

2.6 Empfehlungssystem

- Schaut welche genre der User gut bewertet hat
- Basiert auf Rating-History

3. Business-Logik

3.1 Rating-Regeln

- Ein User kann jedes Media nur einmal bewerten
- Bewertung kann aber später geändert werden
- Beim Update wird die alte Bewertung überschrieben

3.2 Ownership

- Nur wer Media erstellt hat, kann es ändern/löschen
- Wird im Service geprüft
- Wenn falscher User, gibt's einen 403 Forbidden Error

3.3 Kommentar-Freigabe

- Kommentare müssen bestätigt werden bevor sie öffentlich sind
- Ohne die Bestätigung ist die Bewertung nicht sichtbar

3.4 Durchschnitts-Bewertung

- Wird bei jedem neuen Rating neu berechnet
- Alle Ratings für ein Media werden addiert und durch Anzahl geteilt
- Ergebnis wird in Media-Tabelle gespeichert

3.5 Empfehlungs-Algorithmus

- Schaut was die gut fanden
- Empfiehlt Media das der aktuelle User noch nicht kennt

3.6 Leaderboard

- Zeigt Top-User nach Anzahl Ratings
- Oder nach Durchschnitt der Bewertungen die sie bekommen haben

4. SOLID Prinzipien

4.1 Single Responsibility Principle (SRP)

Jede Klasse hat nur eine Aufgabe.

Beispiel im Projekt:

- `UserHandler` : Nur HTTP-Requests für User verarbeiten
- `UserService` : Nur User-Logik (registrieren, login, etc.)
- `UserDAO` : Nur Datenbank-Operationen für User

Warum ist das gut?

- Wenn was kaputt geht, weiß man genau wo man suchen muss
- Kann Sachen ändern ohne andere Teile zu brechen
- Tests sind einfacher zu schreiben

4.2 Dependency Inversion Principle (DIP)

Was bedeutet das? Klassen sollen nicht direkt voneinander abhängen, sondern über Interfaces.

Beispiel im Projekt:

- Service-Klassen bekommen ihre DAOs über den Konstruktor
- Nicht: `this.userDAO = new UserDAO()`
- Sondern: `public UserService(UserDAO userDAO) { this.userDAO = userDAO; }`

Warum ist das gut?

- Bei Tests kann man Fake-DAOs verwenden (Mocking)
- Weniger starke Kopplung zwischen Klassen

5. Unit Tests

5.1 Test-Strategie

Was wird getestet?

- Hauptsächlich Service-Schicht (da ist die Logik)
- DAO-Methoden (Datenbank-Operationen)
- Wichtige Helper-Funktionen

5.2 Test-Beispiele

FavoriteServiceTest:

- Testet ob Favoriten hinzugefügt werden
- Testet ob Favoriten entfernt werden
- Testet ob Liste korrekt zurückgegeben wird

UserServiceTest: - Testet die Bearbeitung der Profile

AuthServiceTest: - Testet Registrierung - Testet Login mit richtigem/falschem Passwort

TokenHelperTest:

- Testet Token-Validierung

RatingServiceTest:

- Testet Rating erstellen
- Testet Rating updaten
- Testet ob User nur einmal raten kann

5.3 Test-Abdeckung

- Mindestens 20 sinnvolle Unit Tests
- Fokus auf Business-Logik
- Keine Duplikate
- Jeder Test prüft genau eine Sache

6. Sicherheit

6.1 SQL-Injection Prevention

Problem: Wenn User-Input direkt ins SQL kommt, könnte jemand schädlichen Code einschleusen.

Lösung:

- Prepared Statements verwenden
- User-Input wird als Parameter übergeben
- Datenbank escaped die Werte automatisch

6.2 Token-basierte Authentifizierung

- Passwörter werden gehasht gespeichert (nicht im Klartext)
- Nach Login bekommt User einen Token
- Token ändert sich nach jedem Login
- Token wird als Bearer Token verwendet

7. Datenbank und Deployment

7.1 PostgreSQL Setup

- Datenbank läuft in Docker-Container
- Port 5432 wird exposed
- Credentials in Docker Compose definiert

7.2 Migrations

- SQL-Scripts für Tabellen-Erstellung
- Scripts für Test-Daten
- Alles versioniert im Git

8. Integration Tests

- Postman Collection mit allen Endpoints
- Testet komplette Workflows (registrieren, login, media erstellen, bewerten)

9. Lessons Learned

9.1 Was lief gut?

- Drei-Schichten-Architektur macht Sinn
- Prepared Statements sind easy zu verwenden
- Token-Auth ist simpel aber effektiv
- Tests helfen Bugs früh zu finden

9.2 Was war schwierig?

- Komplexe Joins für Empfehlungen waren tricky
- Transaction-Handling bei mehreren Tabellen
- Fehlerbehandlung konsistent machen

9.3 Was würde ich anders machen?

- Früher mit Tests anfangen
- Mehr Zeit für Datenbank-Design nehmen
- Besseres Error-Handling von Anfang an (Custom Exceptions erstellen)

10. Zeitaufwand

Aufgabe	Zeit (ca.)
Setup & Infrastruktur	6h
User Management	8h
Media Management (CRUD)	12h
Rating-System	10h
Favoriten	4h
Empfehlungssystem	6h
Leaderboard	3h
Helper & Utilities	4h
Unit Tests schreiben	5h
Debugging & Refactoring	8h
Dokumentation	4h
Gesamt	~70h

11. REST-API Übersicht

Authentifizierung:

- POST /register - Neuen User registrieren
- POST /login - User einloggen, bekommt Token zurück

User-Management:

- GET /users/{userId}/profile - User-Profil anzeigen
- POST /users/{userId}/profile - User-Profil bearbeiten
- PUT /users/{userId}/profile - User-Profil aktualisieren
- GET /users/{userId}/ratings - Rating-History eines Users anzeigen
- GET /users/{userId}/favorites - Favoriten eines Users anzeigen

Media:

- GET /media - Alle Media anzeigen (mit Query-Parametern für Filter/Suche)

- GET /media/{mediald} - Ein bestimmtes Media anzeigen
- POST /media - Neues Media erstellen
- PUT /media/{mediald} - Media bearbeiten (nur Creator)
- DELETE /media/{mediald} - Media löschen (nur Creator)

Ratings:

- GET /ratings/{mediald} - Alle Ratings für ein Media anzeigen
- POST /ratings/{mediald}/rate - Rating für ein Media erstellen/updaten
- PUT /ratings/{mediald}/rate - Rating für ein Media aktualisieren
- DELETE /ratings/{ratingId}/rate - Rating löschen
- POST /ratings/{ratingId}/like - Rating liken
- DELETE /ratings/{ratingId}/like - Like von Rating entfernen
- POST /ratings/{ratingId}/approve - Rating-Kommentar freigeben (Admin/Moderator)

Favorites:

- GET /favorites - Alle Favoriten des eingeloggten Users
- POST /favorites/{mediald} - Media zu Favoriten hinzufügen
- DELETE /favorites/{mediald} - Media aus Favoriten entfernen

Empfehlungen & Leaderboard:

- GET /recommendations - Empfohlene Media basierend auf Rating-History
- GET /leaderboard - User-Leaderboard (Top-Rater)